



SCHOOL OF FUTURE TECH

ITM Skills University

CASE STUDY REPORT

MCL Studio

167 – Markdown Component Layout Studio

Submitted by

Aaryan Kuchekar

Enrollment No: 150096725048

B.Tech – Computer Science & Engineering

ITM Skills University

GitHub: github.com/Aaryan0612

1. Abstract

MCL Studio is a frontend-only React and Vite application that transforms a constrained markdown-like language into a validated layout canvas. Instead of rendering directly from raw text, the application tokenizes user input, builds an Abstract Syntax Tree (AST), validates structural rules, generates diagnostics, and renders the result as a synchronized visual workspace. The system was built to demonstrate both product usability and engineering depth through AST inspection, graph visualization, export support, and browser persistence.

2. Project Explained Simply

Traditional markdown is useful for writing headings and paragraphs, but it cannot naturally describe structured layouts such as sidebars, columns, and grouped information blocks. MCL Studio solves this by introducing a small layout language with explicit blocks such as row, col, and info. The user types on the left, the system understands the structure, checks whether it is valid, and renders the result on the right.

1. The user writes layout markdown in the editor.
2. The parser converts the input into tokens.
3. The AST builder converts tokens into a tree.
4. The validator checks nesting and width rules.
5. The renderer draws the validated structure on the layout canvas.

3. Problem Statement

The case study problem was to design a React application that could interpret a markdown-like layout language, generate a structured intermediate representation, validate the structure, and render the result as a live preview. The application also needed enough technical transparency to prove the parser exists and enough usability features to feel complete.

4. Objectives

1. Build a structured markdown-like layout grammar.
2. Tokenize user input with line and column metadata.
3. Build an AST from the token stream.
4. Validate width and structural nesting rules.
5. Render the AST as a live layout canvas.
6. Expose developer tools such as AST inspector and AST graph view.
7. Support persistence and AST export.

5. Technology Stack

Area	Technology	Reason
Frontend	React 19	Component-driven UI and state orchestration
Build Tool	Vite 8	Fast development and clean static build
Language	JavaScript	Lightweight implementation aligned to scope
Testing	Vitest / RTL / Playwright	Validation across unit, integration, and E2E layers
Deployment	Vercel	Static hosting for Vite frontend
Persistence	localStorage	Restore workspace content across reloads

6. React Concepts Demonstrated

Concept	Usage in Project
Components	EditorPanel, PreviewPanel, ASTInspector, AppHeader, DiagnosticsBar
Props	Parent-child communication from AppShell into editor, preview, and AST tools
useState	rawText, selectedNodeId, activeTemplateId, exportState, layout boundary mode
useEffect	Persistence updates, debounce timing, and editor source-focus synchronization
Custom Hooks	useParser, useLocalStorage, useDebounceValue
Controlled Components	The main editor textarea is controlled by React state

Concept	Usage in Project
Conditional Rendering	Preview empty state, graph/outline mode, diagnostics state changes
Local Storage	Workspace persistence using browser storage

7. System Architecture

MCL Studio is built as a parser-driven frontend application. Raw text does not become preview UI directly. Instead, the system uses an explicit parsing pipeline and only renders from the AST.

Frozen Grammar Rules

- ROOT can contain ROW, INFO, headings, and text.
- ROW can contain COL only.
- COL can contain H1, H2, H3, TEXT, and INFO only.
- Nested ROW inside COL is invalid in version 1.

Grammar Specification

```
ROOT  -> ROW | INFO | H1 | H2 | H3 | TEXT
ROW   -> COL
COL   -> H1 | H2 | H3 | TEXT | INFO
INFO  -> H1 | H2 | H3 | TEXT
```

Grammar Usage Example

```
:::row
  :::col width=100
    # Welcome
    Describe your layout.
  :::
:::
```

AST Example

```
root
├─ row
│   └─ col
│       ├── H1
│       └─ text
```

The AST stores node type, children, source line range, and width or content metadata where appropriate. This makes validation, graph visualization, and renderer consistency possible.

Complexity Analysis

Operation	Time Complexity
Tokenization	$O(n)$
AST Building	$O(n)$
Validation	$O(n)$
Rendering	$O(n)$

8. Component Architecture

AppShell is the central coordinator. It owns shared state such as raw text, selected node, template selection, export status, and parser results. Child components focus on rendering or signaling user intent.

Key Components

- AppShell — Root coordinator; owns all shared state
- EditorPanel — Controlled textarea for layout markdown input
- PreviewPanel — Renders AST into live layout canvas
- ASTInspector — Outline view of the parsed tree nodes
- ASTGraphView — Visual graph of AST structure
- DiagnosticsBar — Real-time error and warning feed
- AppHeader — Status and global action surface

State Ownership

- rawText — Editor content string
- selectedNodeId — Active AST node for cross-panel sync
- activeTemplateId — Selected starter template
- exportState — Export lifecycle flag
- parseResult — Memoized parser output (tokens, AST, diagnostics)

9. Features Implemented

- Structured layout editor
- Tokenizer with source metadata (line and column numbers)
- AST builder with source ranges and node IDs
- Structural validator for nesting and width rules
- Diagnostics summary with error and warning counts
- Live layout canvas from validated AST
- AST outline inspector with node selection
- AST graph view for visual tree exploration
- Template loader with starter examples
- AST export to ast.json
- Browser persistence via localStorage

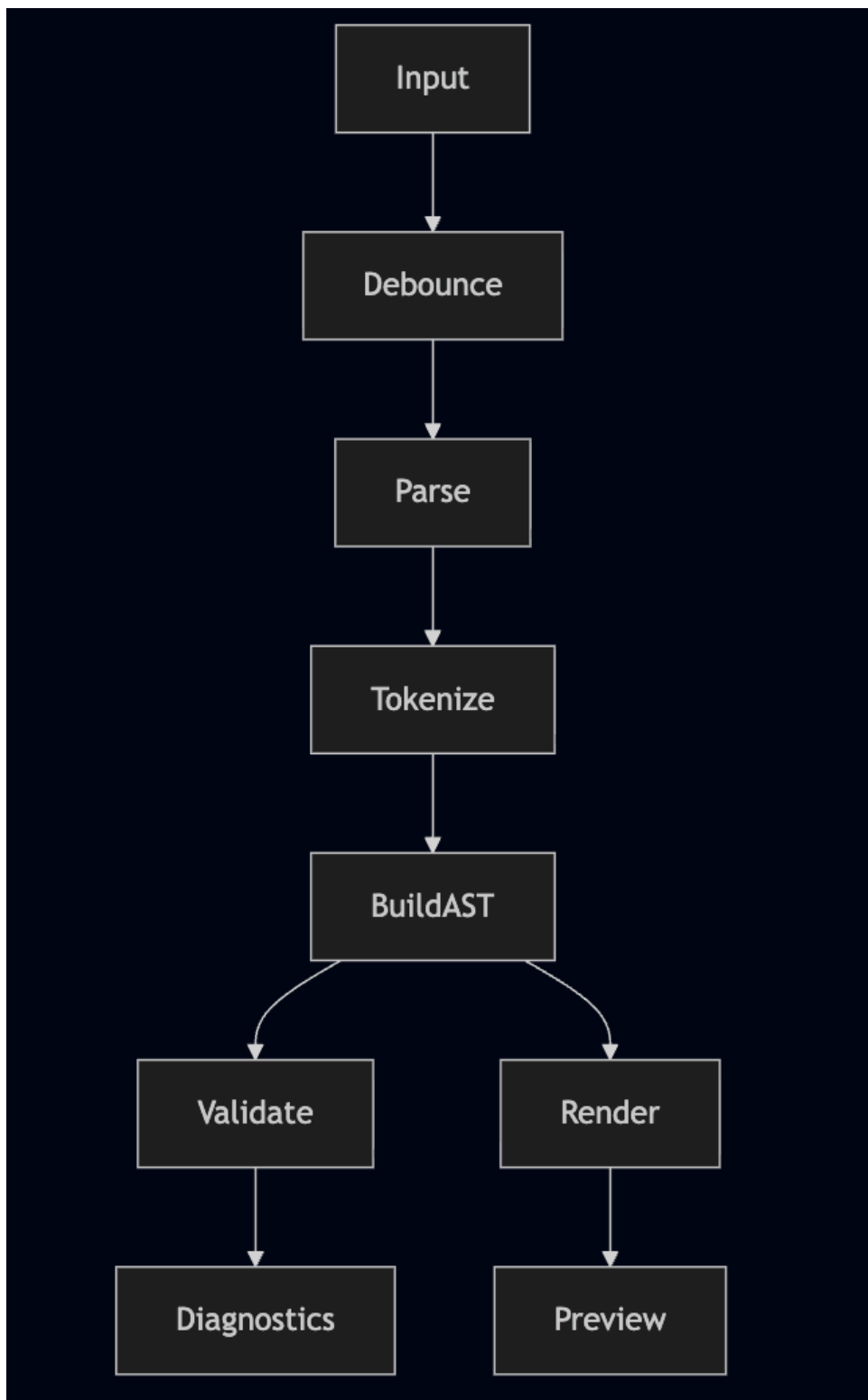
10. Requirement Mapping

Requirement	Implementation
Markdown Editor	src/components/editor/EditorPanel.jsx
Live Preview	src/components/preview/PreviewPanel.jsx
AST Generation	src/parser/astBuilder.js
Validation	src/parser/validator.js
Diagnostics	src/utls/diagnostics.js
Persistence	src/hooks/useLocalStorage.js
Export	handleExportAst() in src/components/layout/AppShell.jsx
AST Visualization	src/components/ast/ASTInspector.jsx and ASTGraphView.jsx

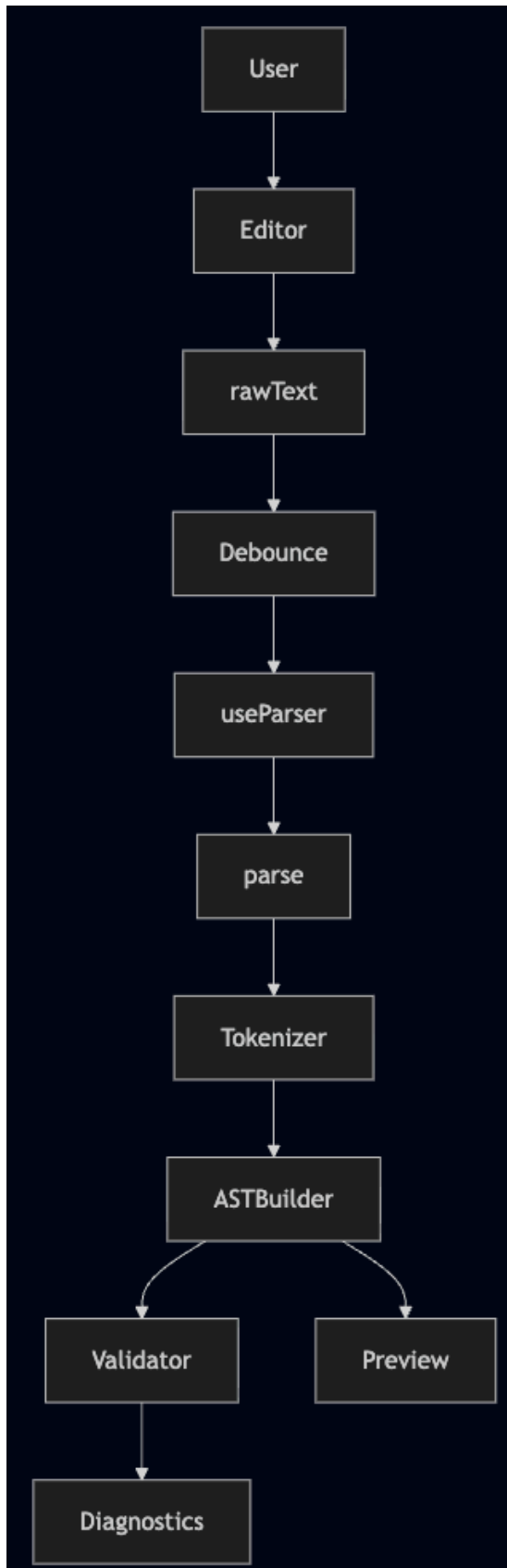
11. Implementation Details

The application uses a debounced parser flow. The editor updates raw text state immediately. After a 150ms debounce, `useParser` invokes `parse()`, which calls `tokenize()`, `buildAST()`, `validateParseArtifacts()`, and `buildDiagnostics()`. The resulting parse result powers the preview, AST tools, header status, and diagnostics footer.

Parsing Pipeline



Data Flow

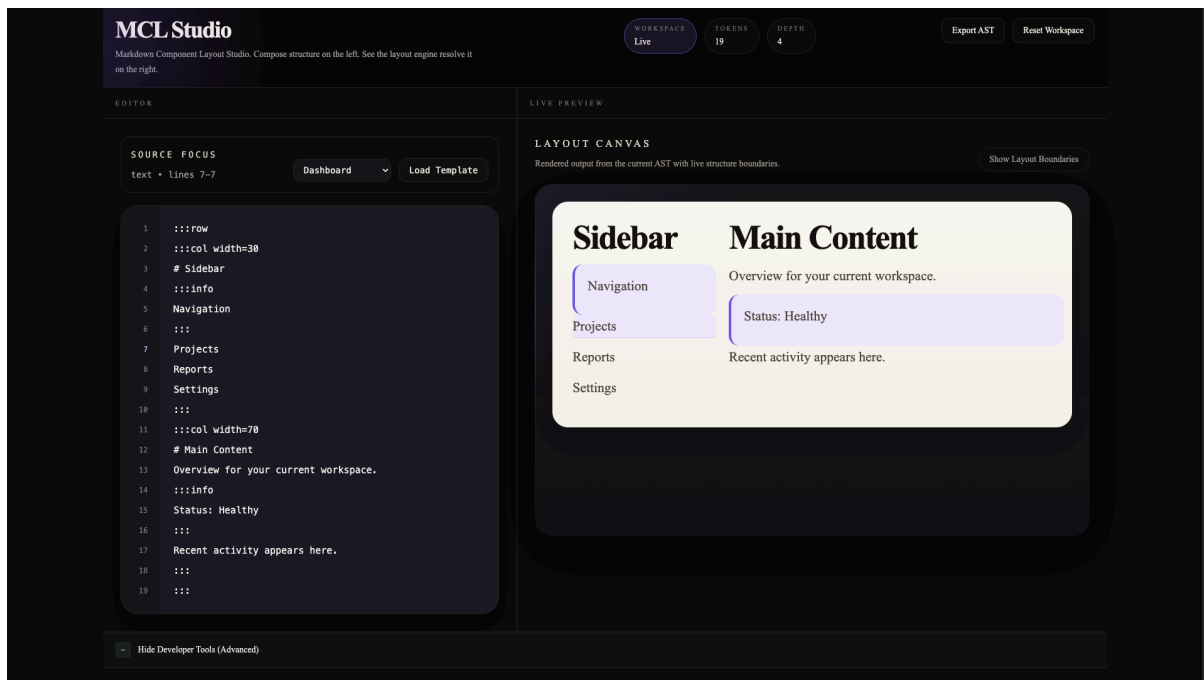


parseResult flows from AppShell (via useParser) into EditorPanel for token highlighting, PreviewPanel for rendering, ASTInspector and ASTGraphView for developer tooling, DiagnosticsBar for error display, and AppHeader for status indicators. All consumers are read-only; only AppShell writes to parse state.

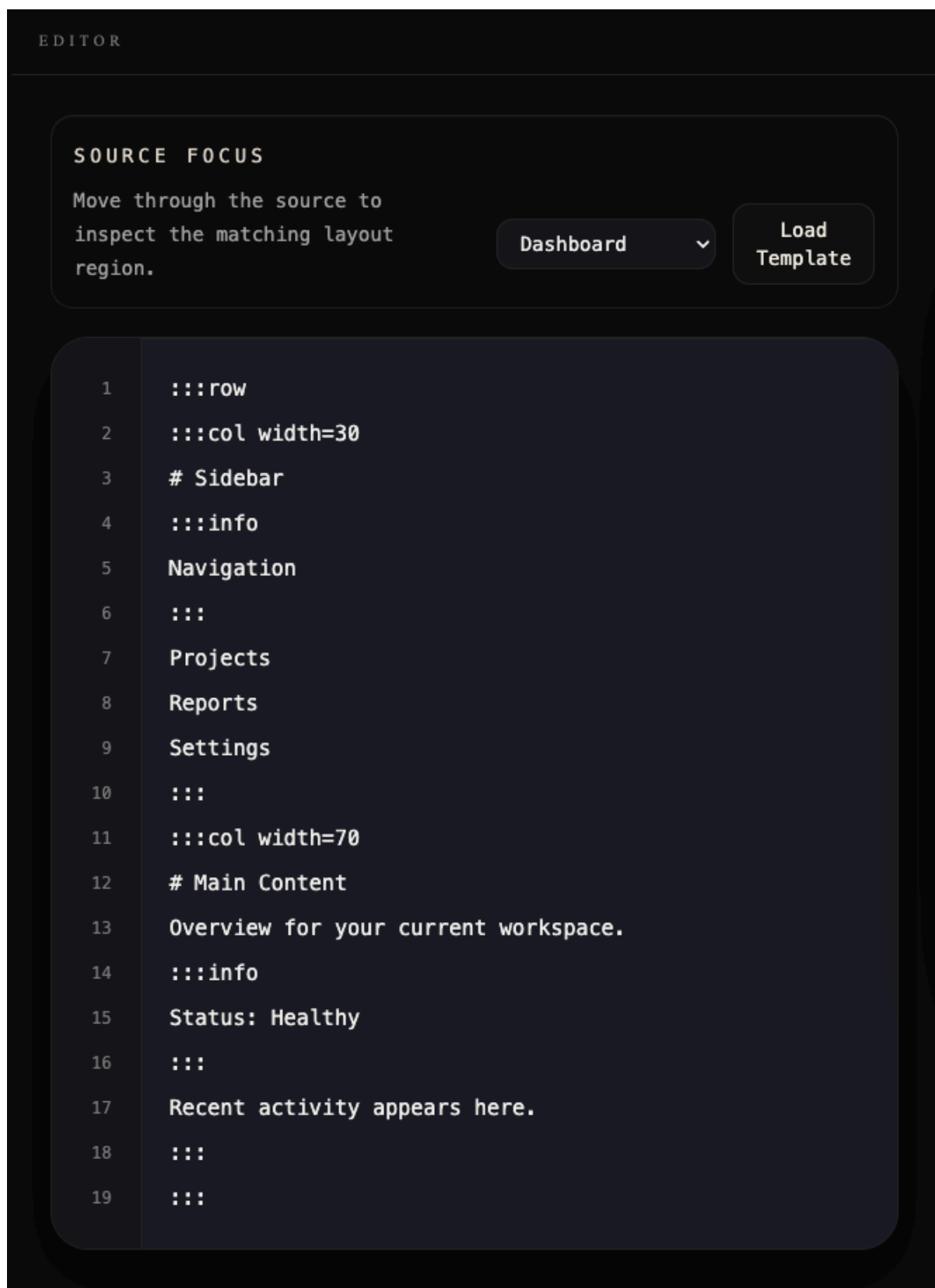
12. Screenshots

The following views are available in the deployed application at mcl-studio.vercel.app:

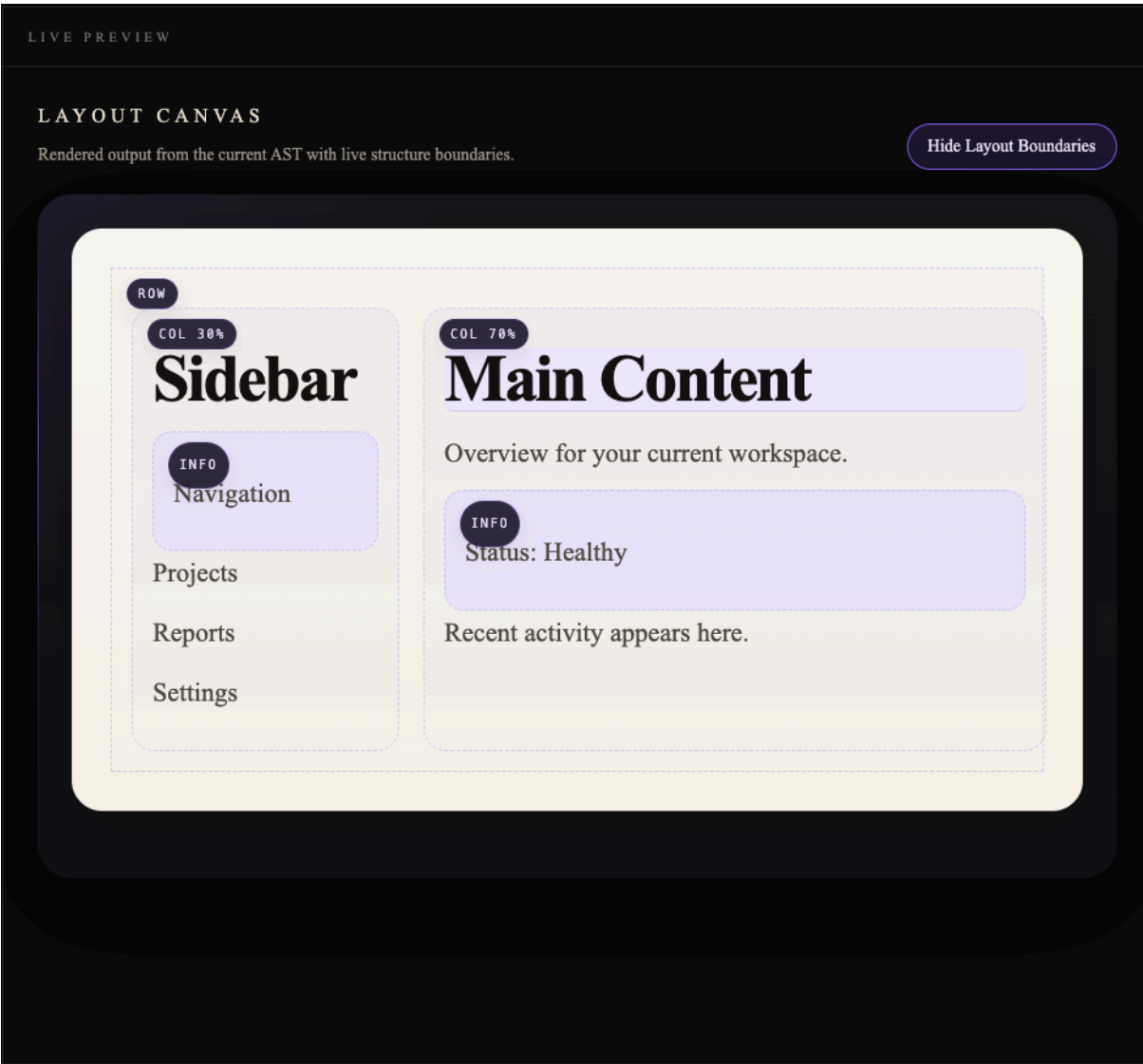
- Main Workspace — full editor + preview split view



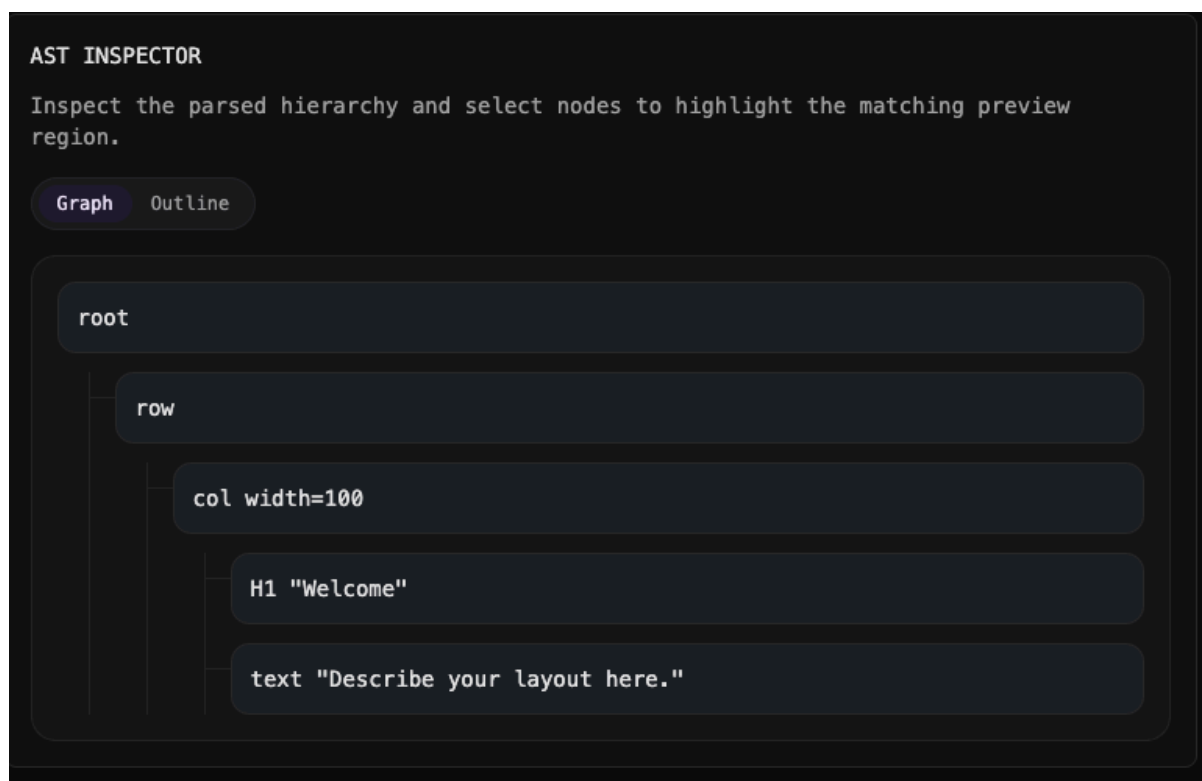
- Editor — controlled textarea with syntax feedback



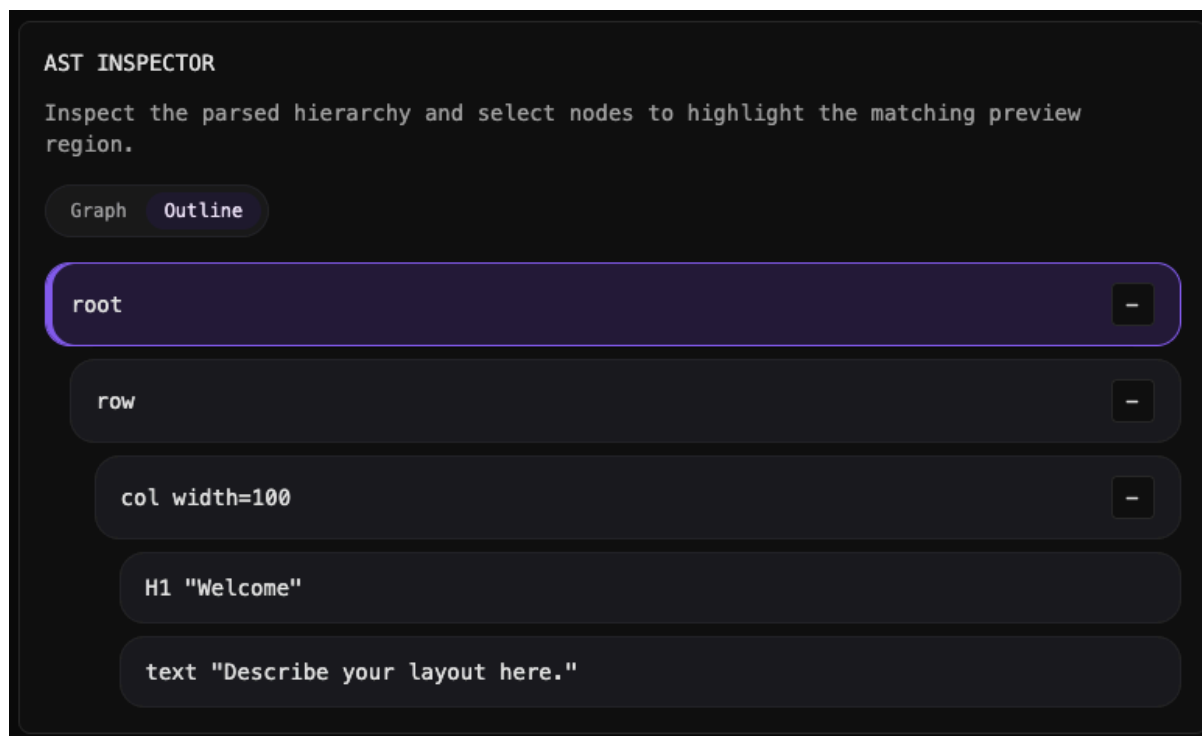
- Preview Canvas — validated AST rendered as layout



- AST Graph View — interactive node graph



- AST Inspector — outline tree with selection sync



- Diagnostics Panel — live error and warning list

System diagnostics

Parser metrics stay read-only and are grouped here for developer inspection.

STATUS

Layout Valid

TOKENS

6

DEPTH

3

PARSE TIME

0.20ms

ERRORS

0

WARNINGS

0

- Template Loader — built-in starter templates

MCL Studio

Markdown Component Layout Studio. Compose structure on the left. See the layout engine resolve it on the right.

WORKSPACE

Live

TOKENS

6

DEPTH

3

Export AST

Reset Workspace

EDITOR

SOURCE FOCUS

root • lines 1-6

Starter

✓ Dashboard

Documentation

Load Template

```
1  :::row
2  :::col width=100
3  # Welcome
4  Describe your layout here.
5  :::
6  :::
```

LIVE PREVIEW

LAYOUT CANVAS

Rendered output from the current AST with live structure boundaries.

Hide Layout Boundaries

ROW

COL 100%

Welcome

Describe your layout here.

- Export Feature — one-click ast.json download

MCL Studio

Markdown Component Layout Studio. Compose structure on the left. See the layout engine resolve it on the right.

WORKSPACE

Live

TOKENS

6

DEPTH

3

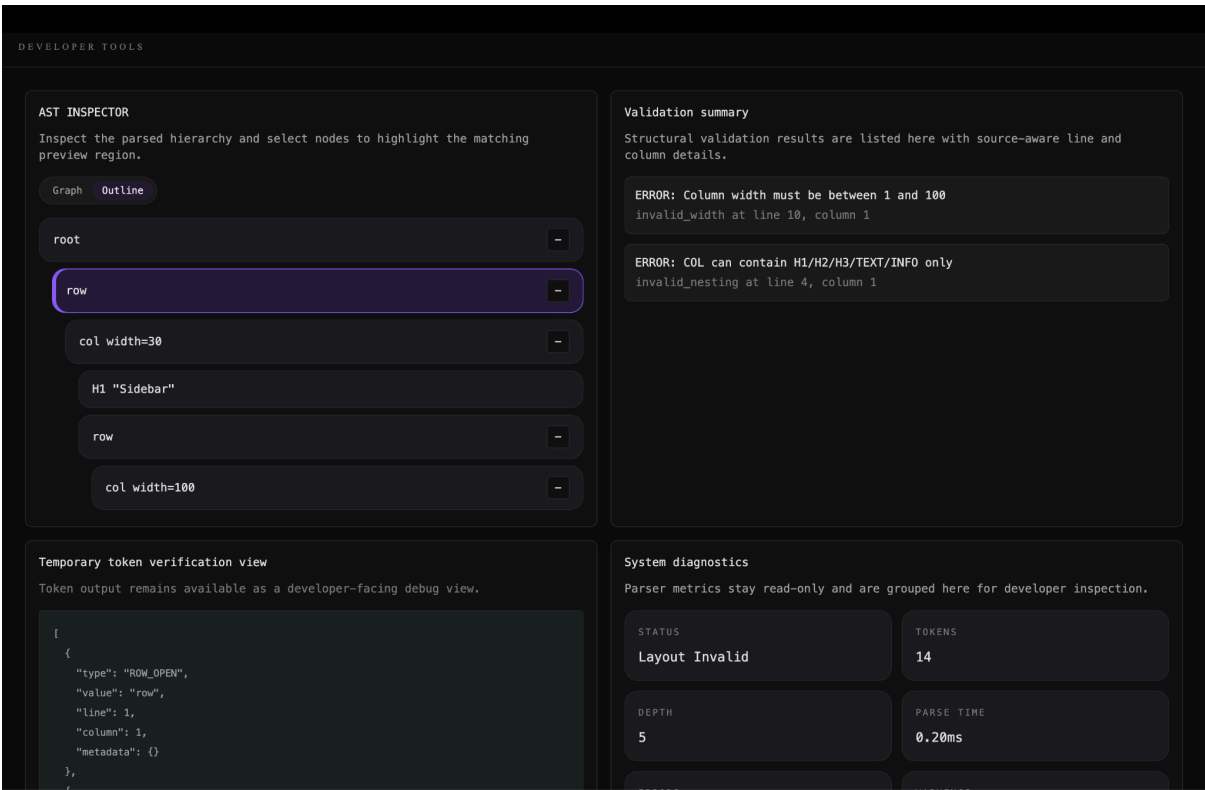
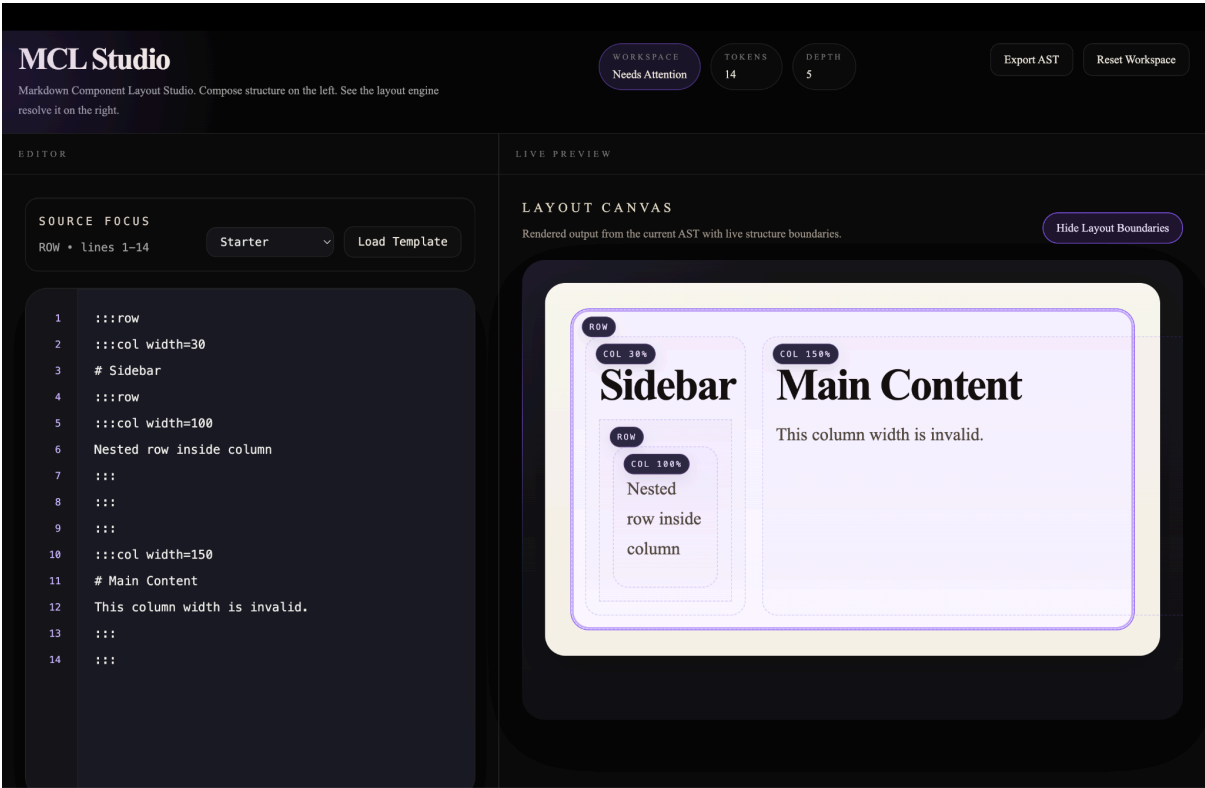
Export AST

Reset Workspace

Aaryan Kuchekar • 150096725048 • B.Tech CSE • ITM Skills University | Page 13

```
{
  "id": "root",
  "type": "root",
  "startLine": 1,
  "endLine": 6,
  "children": [
    {
      "id": "node-1",
      "startLine": 1,
      "endLine": 6,
      "children": [
        {
          "id": "node-2",
          "startLine": 2,
          "endLine": 5,
          "children": [
            {
              "id": "node-3",
              "startLine": 3,
              "endLine": 3,
              "children": [],
              "type": "H1",
              "content": "Welcome"
            },
            {
              "id": "node-4",
              "startLine": 4,
              "endLine": 4,
              "children": [],
              "type": "text",
              "content": "Describe your layout here."
            }
          ],
          "type": "col",
          "width": 100
        },
        {
          "type": "row"
        }
      ]
    }
  ]
}
```

- Validation Errors — inline error markers in diagnostics



13. Competitive Analysis

Tool	Layout Grammar	AST View	Diagnostics	Export AST
Traditional Markdown Previewers	No	No	No	No
Obsidian	Partial	No	No	No
Notion	Different	No	No	No
MCL Studio	Yes	Yes	Yes	Yes

14. Major Engineering Decisions

Why React — React was chosen because the project depends on multiple synchronized views and shared interaction state. Its component model and unidirectional data flow made it the natural fit for coordinating the editor, preview, AST tools, and diagnostics simultaneously.

Why Vite — Vite provided a fast and simple development workflow for a static frontend project. Near-instant HMR and a clean production build configuration reduced tooling overhead.

Why AST — An intermediate AST made validation, graph visualization, export, and rendering consistency all possible without coupling parser logic to UI concerns.

Why localStorage — Browser persistence was sufficient for this scope and avoided introducing unnecessary backend complexity.

Why Custom Grammar — Plain markdown cannot express row-column layout structure clearly enough. A small, constrained grammar gave explicit semantics that the validator and renderer could rely on.

Why Frontend-only — The core value of the project is client-side parsing, validation, and rendering. Server processing would add latency and complexity with no benefit.

15. Challenges Faced During Development

Problem

Parser validation issues — invalid nesting was difficult to detect inline.

Solution & Learning

A dedicated structural validator was built so nesting errors could be detected cleanly and reported separately from render concerns.

Problem

AST synchronization issues — selected nodes became stale across re-parses.

Solution & Learning

Selection safety and node existence checks were centralized in AppShell to prevent stale references.

Problem

Selection mapping problems — containers and leaf nodes behaved inconsistently when highlighted.

Solution & Learning

The renderer introduced node-type-aware highlighting so containers and leaf nodes use different selection semantics.

Problem

Layout fidelity problems — rows and columns did not communicate structure clearly enough.

Solution & Learning

The preview was refined so the layout canvas conveys structure rather than just rendering content.

Problem

Export and persistence — Browser APIs could throw on restricted origins.

Solution & Learning

Browser APIs were wrapped safely so failures do not crash the product and degrade gracefully.

16. Results and Observations

MCL Studio successfully achieved its core goals. It parses structured layout input, builds a source-aware AST, validates structure, renders a live layout canvas, visualizes the AST, supports persistence, and exports ast.json. The final architecture demonstrates strong separation of concerns: parser logic is not embedded in UI components, rendering is AST-driven, and diagnostics are computed separately from display.

17. Conclusion

MCL Studio demonstrates how a React application can combine component architecture with parser-driven design. It is both a usable product and a technical case study. The project shows understanding of React state, hooks, controlled inputs, browser APIs, AST design, structural validation, and live rendering.

18. Future Scope

- Implementation of a refined layout grammar for advanced nesting
- Integration of specialized block primitives beyond standard containers
- Dynamic preview engine with comprehensive theming support
- Standardized workflows for modular template sharing and migration
- Real-time collaboration powered by centralized cloud persistence

19. Repository & Deployment

GitHub Repository:

github.com/Aaryan0612/MCL-Studio

Live Deployment:

mcl-studio.vercel.app

20. References

- React Official Documentation — react.dev
- Vite Official Documentation — vitejs.dev
- Vitest Documentation — vitest.dev
- React Testing Library Documentation — testing-library.com/docs/react-testing-library/intro
- Playwright Documentation — playwright.dev
- Browser APIs — MDN Web Docs (localStorage, Blob, download links)
- MCL Studio repository source files and release documentation